**EEE4120F**

## High Performance Embedded Systems

# REFERENCE GUIDE

Verilog HDL and Icarus Verilog (iverilog)
Practical 4 Tutorial and Setup Guide

### Course Team


**Course Convener:** Associate Professor Simon Winberg

**Teaching Assistant:** Travimadox Webb

# Verilog HDL and Icarus Verilog

A Practical Setup and Reference Guide for Practical 4

## 1 Introduction

This tutorial supports Practical 4 by introducing the Verilog HDL language and the Icarus Verilog (iverilog) simulation toolchain. After working through this guide you will be able to install the tools, compile Verilog source files, run simulations, inspect waveforms in GTKWave, and write testbenches that automatically verify circuit behaviour.

The guide is structured as follows: Section 2 covers tool installation on all three platforms. Section 3 introduces the Verilog language from first principles. Section 4 explains the iverilog and GTKWave workflow. Section 5 covers testbench writing in depth. Section 6 presents a complete worked example you can run immediately to confirm your setup. Section 7 covers common errors and how to diagnose them. Section 8 provides a quick-reference card.

**Note**

This guide focuses on the Verilog-2001 standard, which iverilog supports fully and which is sufficient for Practical 4. Constructs from SystemVerilog (e.g., `logic`, `always_comb`, `always_ff`) are not used in this guide but are mentioned where they provide useful context.

### 1.1 What is Icarus Verilog?

Icarus Verilog (iverilog) is a free, open-source Verilog simulation and synthesis tool maintained by Stephen Williams. It compiles Verilog source files into a simulation executable using an intermediate representation called vvp (Verilog Virtual Processor). The compiled simulation can then be run to produce output (printed via `$display`) and waveform data (in Value Change Dump, or .vcd, format).

The iverilog toolchain consists of two separate programs:

- **iverilog** — the compiler/elaborator. It reads .v files, resolves module hierarchies, and produces a compiled simulation object file.
- **vvp** — the simulation runtime. It executes the compiled object and produces output. When you run ./a.out or ./sim_name, you are actually running vvp behind the scenes.

GTKWave is a waveform viewer that reads .vcd files produced by simulation and displays signal values over time as a timing diagram. It is the primary tool for visual debugging of digital designs.

### 1.2 The Most Important Verilog Mindset Shift

Before installing anything, internalise this: **Verilog is not a programming language. It is a hardware description language.** When you write Verilog, you are not instructing a CPU to execute statements one after another. You are describing concurrent digital hardware — gates, flip-flops, multiplexers, and wires — that all operate simultaneously.

The implications of this are profound:

- Two `always` blocks in the same module run concurrently, not sequentially. Their order in the source file is irrelevant to simulation.
- All continuous `assign` statements are active simultaneously and update their outputs whenever any input changes.
- There is no 'program counter' stepping through your Verilog file. Time advances only via delay constructs (`#N`) and clock edges.
- **Every signal must have exactly one driver.** Two `always` blocks driving the same variable cause a multiple-driver conflict — equivalent to a short-circuit in hardware.

**Warning**

The single most common mistake made by students transitioning from software to hardware design is treating `always` blocks as sequential functions. They are not. Each `always` block describes hardware that reacts to its sensitivity list events; it does not 'run' in a loop after another block finishes.

# 2 Installing iverilog and GTKWave

## 2.1 Linux (Ubuntu 22.04 / 24.04 and Debian)

Most Linux distributions include iverilog in their package manager. Open a terminal and run:

```
sudo apt update
sudo apt install -y iverilog gtkwave
```

Verify the installation:

```
iverilog -v
gtkwave --version
```

Expected output (version numbers may differ):

```
Icarus Verilog version 11.0 (stable) ()
GTKWave Analyzer v3.3.116 (w)2012-2023 BSI
```

**Note**

If gtkwave fails to launch with a display error (cannot open display), you may need to run it on a machine with a graphical environment, or use X11 forwarding if connected via SSH: `ssh -X username@hostname`.

## 2.2 macOS (Homebrew)

On macOS, the system-provided gcc is actually Clang and iverilog must be installed via Homebrew. If you do not have Homebrew installed, visit https://brew.sh for the one-line installation command.

```
brew update
brew install icarus-verilog
brew install --cask gtkwave
```

After installation, verify:

```
iverilog -v
gtkwave --version
```

**Warning**

macOS Gatekeeper may block GTKWave on first launch with a message like 'GTKWave cannot be opened because the developer cannot be verified'. To fix: open System Settings > Privacy & Security, scroll to the Security section, and click 'Open Anyway' next to the GTKWave entry.
On Apple Silicon (M1/M2/M3) Macs, Homebrew installs to `/opt/homebrew/bin`. Ensure this is on your PATH: add `export PATH=/opt/homebrew/bin:$PATH` to your `~/.zshrc` and reload with `source ~/.zshrc`.

## 2.3 Windows — Strongly Recommended: WSL2

The teaching team strongly recommends using WSL2 (Windows Subsystem for Linux) with Ubuntu 22.04 or 24.04. This gives you a full Linux environment inside Windows without dual-booting, and allows you to follow the Linux instructions above exactly.

### 2.3.1 Installing WSL2

1. Open PowerShell as Administrator (right-click Start > Windows PowerShell (Admin)).
2. Run the following command:

```
wsl --install
```

3. When prompted, reboot your machine.
4. After rebooting, Ubuntu will open automatically and ask you to create a Unix username and password. Choose something simple — you will need to type the password for `sudo` commands.
5. Once the Ubuntu prompt appears, follow the Linux installation instructions in Section 2.1.

### 2.3.2 Accessing GTKWave from WSL2

On Windows 11, WSLg (Windows Subsystem for Linux GUI) is built in and GUI applications like GTKWave launch automatically from the WSL terminal. On Windows 10, you need an X server:

1. Download and install VcXsrv from https://sourceforge.net/projects/vcxsrv/
2. Launch VcXsrv with the XLaunch wizard; select 'Multiple windows' and 'Disable access control'.
3. In your WSL terminal, set the DISPLAY variable:

```
export DISPLAY=$(cat /etc/resolv.conf | grep nameserver | awk '{print
$2}'):0
```

4. Test by running: `gtkwave --version`

**Tip**

Alternatively, copy the .vcd file from WSL to your Windows filesystem and open it with GTKWave for Windows (available at gtkwave.sourceforge.net). Your WSL home directory is accessible from Windows Explorer at

`wsl$`
`Ubuntu\home\yourusername\`

### 2.3.3 Native Windows (Not Recommended)

A native Windows build of iverilog exists but is not officially supported by the teaching team. Path handling differences, file encoding issues, and the lack of the mpicc-style wrapper make debugging time-consuming. If you choose this path, download the Icarus Verilog Windows installer from http://bleyer.org/icarus/ and add the installation's bin directory to your Windows PATH. GTKWave for Windows is available from the project website.

## 2.4 Confirming Your Installation Works

Create the following file named `hello.v` in any directory:

```
// hello.v -- installation verification
module hello;
initial begin
    $display("iverilog is working! Time = %0t", $time);
    $finish;
end
endmodule
```

Compile and run:

```
iverilog -o hello hello.v
./hello
```

Expected output:

```
iverilog is working!  Time = 0
```

If you see this, your installation is correct. Proceed to Section 3.

# 3 Verilog HDL Fundamentals

## 3.1 Modules

The `module` is the fundamental unit of Verilog. Every component you design — an ALU, a register file, a memory — is a module. Modules communicate with the outside world through ports. Larger modules are built by instantiating smaller modules and wiring their ports together.

The general module structure is:

```
module ModuleName (
    // Port declarations
    input [width-1:0] port_name,
    output [width-1:0] port_name
);

    // Internal signal declarations
    // Continuous assignments
    assign ...;

    // Procedural blocks
    always @(*) begin ... end
    always @(posedge clk) begin ... end

    // Sub-module instantiations
    SubModule inst_name ( .port(wire), ... );

endmodule
```

**Note**

Every Verilog file must end with `endmodule`. A common compile error is missing this keyword.
One module per file is a strong convention (though not enforced by iverilog). Name each file after its module: `ALU.v` contains `module ALU`.

## 3.2 Data Types: wire and reg

Verilog has two primary data types for describing hardware signals. Understanding the difference between them is essential:

| Type | Declare when... | Hardware Model |
|---|---|---|
| `wire` | driven by `assign` or a sub-module output port | A physical wire; value determined continuously by driver |
| `reg` | assigned inside an `always` block | A storage element (flip-flop or register) |

**Warning**

Despite its name, `reg` does NOT always mean a hardware register (flip-flop). A `reg` assigned inside `always @(*)` models combinational logic — a lookup table or mux — not a flip-flop.
The hardware a `reg` becomes depends entirely on its `always` block's sensitivity list: `@(posedge clk)` creates a flip-flop; `@(*)` creates combinational logic.

Declaring signals:

```
wire single_bit_wire;                // 1-bit wire
wire [7:0] byte_bus;                 // 8-bit wire bus
reg [15:0] sixteen_bit_register;     // 16-bit reg
reg [15:0] reg_array [7:0];          // Array of 8 x 16-bit registers
```

## 3.3 Continuous Assignments

The `assign` keyword creates a permanent, continuous connection. The left-hand side (LHS) must be a `wire`. The right-hand side (RHS) is re-evaluated instantaneously whenever any of its operands change. Think of `assign` as wiring two points together with combinational logic in between.

```
assign sum = a + b;                              // Adder
assign zero = (result == 16'd0);                 // Comparator
assign out = sel ? a : b;                        // 2-to-1 multiplexer
assign carry_out = a[7] & b[7];                  // Bit selection and AND
    gate
assign ext_im = {{10{instr[5]}}, instr[5:0]}; // Sign extension
```

The last example demonstrates two important operators: the replication operator `{N{x}}` repeats x N times, and the concatenation operator `{a, b}` joins two values. Together, `{{10{instr[5]}}, instr[5:0]}` sign-extends a 6-bit value to 16 bits by replicating the MSB 10 times.

## 3.4 Procedural Blocks

Procedural blocks describe hardware whose behaviour is triggered by events. There are two forms you will use in Practical 4.

### 3.4.1 Combinational Logic: always @(*)

The sensitivity list `@(*)` means 'evaluate whenever any signal on the right-hand side of any assignment in this block changes'. This models combinational (unclocked) logic. Always use **blocking assignments** (=) here.

Example: ALU with a case statement

```
always @(*) begin
    case (alu_control)
        3'b000: result = a + b;              // ADD
        3'b001: result = a - b;              // SUB
        3'b010: result = ~a;                 // INV (one's complement)
        3'b011: result = a << b[3:0];        // SHL
        3'b100: result = a >> b[3:0];        // SHR
        3'b101: result = a & b;              // AND
        3'b110: result = a | b;              // OR
        3'b111: result = (a < b) ? 16'd1 : 16'd0; // SLT
        default: result = a + b;             // Safe default
    endcase
end
```

**Tip**

Always assign default values to all outputs at the top of your `always @(*)` control unit block before the case statement. This avoids accidental latches (unintended memory elements) caused by outputs not being assigned in every case branch.

### 3.4.2 Synchronous (Clocked) Logic: always @(posedge clk)

The sensitivity list `@(posedge clk)` means 'evaluate only on the rising edge of the clock signal'. This models a flip-flop or register. Always use **non-blocking assignments** (<=) here.

Example: Program Counter register

```
always @(posedge clk) begin
    pc_current <= pc_next;  // Updates on every rising clock edge
end
```

Example: Register file with synchronous write

```
always @(posedge clk) begin
    if (reg_write_en)
        reg_array[reg_write_dest] <= reg_write_data;
end
```

### 3.4.3 Why Non-Blocking (¡= ) for Clocked Logic?

Non-blocking assignments compute all RHS values first (using current signal values), then update all LHS values simultaneously at the end of the time step. This correctly models the behaviour of flip-flops where all registers capture their input values at the same clock edge before any output is updated.

If you used blocking (=) in a clocked `always` block, assignments would execute sequentially and a register's updated value could be used as an input to another register in the same block — creating a transparent latch chain instead of a proper register stage.

**Warning**

**Summary of the assignment rules — commit these to memory:**

- `always @(*) { use = (blocking) }`
- `always @(posedge clk) { use <= (non-blocking) }`
- Never mix both styles in the same `always` block
- Never use <= in `always @(*)`
- Never use = for state registers in `always @(posedge clk)`

## 3.5 Number Literals

Verilog number literals have the form: `size'base value`

| Literal | Meaning |
|---|---|
| `16'd42` | 16-bit decimal value 42 |
| `16'hFFFF` | 16-bit hexadecimal 0xFFFF (= 65535) |
| `8'b1010_1010` | 8-bit binary 0xAA (underscores ignored) |
| `4'o7` | 4-bit octal 7 |
| `1'b1` | 1-bit value 1 (logic high) |
| `1'b0` | 1-bit value 0 (logic low) |
| `16'd0` | 16-bit zero (preferred over just 0 for clarity) |

**Note**

Always specify the bit width in literals to avoid implicit width changes. Writing `assign result = a + 1;` when result is 16 bits works, but adding `16'd1` makes the intent explicit and avoids width-mismatch warnings.

## 3.6 Module Instantiation

To compose larger modules from smaller ones, you instantiate sub-modules and connect their ports using wires. Always use named port connections, never positional. Named connections are explicit, refactoring-safe, and far easier to debug.

### 3.6.1 Named Port Connection Syntax

```
ModuleName instance_name (
    .port_name_in_module (wire_name_in_parent),
    .another_port (another_wire),
    .output_port (result_wire)
);
```

Example: Instantiating the ALU inside the Datapath

```
wire [15:0] alu_out;
wire zero_flag;

ALU alu_unit (
    .a (reg_read_data_1),
    .b (alu_operand_b),
    .alu_control (alu_ctrl_signal),
    .result (alu_out),
    .zero (zero_flag)
);
```

### 3.6.2 Wiring Rules

- A wire connected to a module output port is driven by that module. Do not also assign that wire with `assign` — it would create a second driver.
- A wire connected to an input port is driven by the parent module. The module reads its value but does not drive it.
- Unconnected input ports are driven to X (unknown) by default. Always connect all ports.

## 3.7 Parameters and the ìnclude Directive

The `include` directive inserts the contents of another file at the point of inclusion, like `#include` in C. Use it to share parameter definitions across all your modules:

```
`include "../src/Parameter.v"
```

Parameters defined with `define` are text substitution macros:

```
`define COL 16     // Word width
`define ROW_I 16   // Instruction memory depth

// Usage:
reg [`COL-1:0] memory [`ROW_I-1:0];
// Expands to: reg [15:0] memory [15:0];
```

## 3.8 System Tasks

System tasks are built-in simulator functions prefixed with $. They are used in testbenches and simulation code. They are NOT synthesisable hardware constructs — never use them in design files.

| Task | Purpose and Usage |
|---|---|
| `$display("fmt", ...)` | Print formatted output to terminal (like printf) |
| `$monitor("fmt", ...)` | Continuously print when listed signals change |
| `$time` | Return current simulation time as integer |
| `$finish` | End simulation immediately |
| `$dumpfile("f.vcd")` | Open a VCD waveform output file |
| `$dumpvars(0, mod)` | Dump all signals in module to VCD file |
| `$readmemb("f", m)` | Read binary values from file into memory array |
| `$readmemh("f", m)` | Read hexadecimal values from file into memory |

# 4 The iverilog Simulation Workflow

## 4.1 Step 1: Compile

The `iverilog` command parses your Verilog source files and elaborates the module hierarchy, producing a compiled simulation executable (by default named `a.out`). The `-o` flag specifies the output file name.

**Single file:**

```
iverilog -o sim_out MyModule.v MyModule_tb.v
```

**Multiple source files (recommended for Practical 4):**

```
iverilog -Wall -I src/ -o build/alu_sim src/ALU.v tb/ALU_tb.v
```

**All source files at once (for integration test):**

```
iverilog -Wall -I src/ -o build/star_sim src/.v tb/StarCore1_tb.v
```

Important flags:

| Flag | Effect |
|---|---|
| `-o <name>` | Name the compiled output file. Without this, output is `a.out`. |
| `-I <path>` | Add directory to include search path (for `include`) |
| `-Wall` | Enable all warnings. Catches bugs like undeclared wires. |
| `-g2012` | Enable SystemVerilog 2012 extensions. |
| `-v` | Verbose output: prints each file as compiled. |

**Warning**

If iverilog produces errors, fix all of them before attempting to run the simulation. A partially compiled output file may not exist or may produce incorrect results.
**Common compile errors:** 'Unknown module type' means you forgot to include the source file for that module. 'variable declared twice' means you have a duplicate declaration. 'conflicting declaration' means you declared something as both wire and reg.

## 4.2 Step 2: Run the Simulation

```
./sim_out
```

This runs the vvp interpreter on your compiled simulation. All `$display` output appears in the terminal. The simulation runs until `$finish` is called.

If your simulation hangs (does not return to the terminal prompt), it is almost certainly because:

- You forgot `$finish` in the testbench.
- A clocked `always` block runs forever because there is no time limit.
- The program in instruction memory has an infinite JMP loop and `SIM_TIME` is set too long.

Press Ctrl+C to kill a hanging simulation.

## 4.3 Step 3: View Waveforms in GTKWave

If your testbench calls `$dumpfile` and `$dumpvars`, the simulation produces a .vcd file. Open it:

```
gtkwave waves/star.vcd &
```

The & runs GTKWave in the background so your terminal remains usable.

### 4.3.1 Adding Signals to the Waveform View

1. In the SST panel (top-left), click the triangle next to the testbench module name to expand the hierarchy.
2. Click a module name to see its signals in the Signals panel (bottom-left).
3. Double-click a signal name to add it to the waveform display, or drag and drop multiple signals.
4. Press Shift+Ctrl+Home (or Ctrl+Shift+End) to zoom to fit all displayed signal data.
5. Use the scroll wheel or the +/- buttons to zoom in and out on the time axis.

### 4.3.2 Useful GTKWave Tips

- Right-click a signal in the waveform panel to change its display format (binary, decimal, hex, etc.).
- File > Write Save File saves your signal selection to a .gtkw file. Reload with: `gtkwave dump.vcd signals.gtkw`
- Use Edit > Insert Blank to add spacing between groups of signals for readability.
- Markers: click anywhere on the waveform to place a primary marker. Shift-click for a secondary marker. The time difference appears in the toolbar.

# 5 Writing Comprehensive Testbenches

## 5.1 Testbench Structure

A testbench is a Verilog module with no ports that instantiates the Device Under Test (DUT), drives its inputs using `reg` variables, observes its outputs via `wire` variables, and verifies the outputs against expected values. Testbenches are never synthesised.

The basic structure is:

```
`timescale 1ns / 1ps

module ModuleName_tb;

    // 1. DUT inputs --- declared as reg so we can drive them
    reg [15:0] a, b;
    reg [2:0] alu_control;

    // 2. DUT outputs --- declared as wire; driven by the DUT
    wire [15:0] result;
    wire zero;

    // 3. Instantiate the DUT with named port connections
    ALU uut (
        .a(a), .b(b), .alu_control(alu_control),
        .result(result), .zero(zero)
    );

    // 4. Waveform dump (always include this)
    initial begin
        $dumpfile("./waves/alu_tb.vcd");
        $dumpvars(0, ALU_tb);
    end

    // 5. Stimulus and checking
    initial begin
        // apply inputs, wait, then check outputs
        $finish;
    end

endmodule
```

## 5.2 Driving Inputs and Checking Outputs

For combinational modules (like the ALU), apply inputs and wait a short time for outputs to settle before checking. The `#10` delay advances simulation time by 10 ns:

```
initial begin

    // Test ADD: 10 + 5 = 15
    a = 16'd10; b = 16'd5; alu_control = 3'b000;
    #10;  // Wait 10 ns for combinational logic to settle

    if (result !== 16'd15)
        $display("FAIL: ADD expected=15 got=%0d time=%0t", result,
            $time);
    else
        $display("PASS: ADD 10+5 = %0d", result);

```

```
    // Test zero flag: 7 - 7 = 0
    a = 16'd7; b = 16'd7; alu_control = 3'b001;  // SUB
    #10;

    if (zero !== 1'b1)
        $display("FAIL: zero flag not set for result=0");
    else
        $display("PASS: zero flag correctly asserted");

    $display("--- Test complete ---");
    $finish;

end
```

## 5.3 Generating a Clock Signal

For synchronous modules (GPR, DataMemory, and the full processor), your testbench must generate a clock. The standard pattern is:

```
reg clk;

initial clk = 1'b0;  // Initialize before the always block starts
always #5 clk = ~clk; // Toggle every 5 ns -> 10 ns period (100 MHz)
```

Then wait for clock edges in your stimulus:

```
@(posedge clk);   // Wait for the next rising edge
@(negedge clk);   // Wait for the next falling edge
#1;               // Small delay after edge (avoids setup/hold race)
```

**Note**

It is good practice to drive synchronous inputs slightly after the rising edge to avoid setup-time violations in simulation. Use: `@(posedge clk); #1; data_in = new_value;` This ensures the input is sampled correctly by the next rising edge.

## 5.4 Automated Pass/Fail Checking

Relying only on waveform inspection is tedious and error-prone. Use a failure counter to produce a clear summary at the end of each testbench. This makes it easy to run regression tests after modifying your code.

```
integer fail_count;
integer test_num;

initial fail_count = 0;

// Macro-style checking task:
task check;
    input [15:0] got;
    input [15:0] expected;
    input integer test_id;
    begin
        if (got !== expected) begin
            $display("FAIL [Test %0d]: got %0d (0x%h), expected %0d (0
                x%h)",
                test_id, got, got, expected, expected);
            fail_count = fail_count + 1;
```

```
        end else begin
            $display("PASS [Test %0d]: %0d", test_id, got);
        end
    end
endtask

initial begin
    test_num = 1;

    a=16'd10; b=16'd5; alu_control=3'b000; #10;
    check(result, 16'd15, test_num); test_num = test_num + 1;

    a=16'hFFFF; b=16'd1; alu_control=3'b000; #10;
    check(result, 16'd0, test_num); test_num = test_num + 1;  // Overflow wraps

    if (fail_count == 0)
        $display("\n=== ALL %0d TESTS PASSED ===", test_num-1);
    else
        $display("\n=== %0d / %0d TESTS FAILED ===", fail_count, test_num-1);

    $finish;
end
```

**Tip**

Use the `!==` operator (case-inequality) rather than `!=` for checking multi-bit values. The `!==` operator correctly treats X (unknown) and Z (high-impedance) as distinct values.

# 6 Complete Worked Example: 4-bit Full Adder

The following complete, runnable example demonstrates every concept covered in this guide. Study it, type it, compile it, and run it before starting Practical 4.

## 6.1 Design Module: adder4.v

```
`timescale 1ns / 1ps

// 4-bit full adder using carry-ripple structure
module adder4 (
    input [3:0] a,
    input [3:0] b,
    input cin,
    output [3:0] sum,
    output cout
);

    // Concurrent (parallel) addition --- all bits computed
       simultaneously
    assign {cout, sum} = a + b + cin;

endmodule
```

## 6.2 Testbench: adder4_tb.v

```
`timescale 1ns / 1ps

module adder4_tb;

    reg [3:0] a, b;
    reg cin;
    wire [3:0] sum;
    wire cout;

    integer fail_count;
    initial fail_count = 0;

    // Instantiate the DUT
    adder4 uut (.a(a), .b(b), .cin(cin), .sum(sum), .cout(cout));

    // Waveform dump
    initial begin
        $dumpfile("adder4.vcd");
        $dumpvars(0, adder4_tb);
    end

    // Stimulus
    initial begin
        $display("--- adder4 testbench ---");

        // Test 1: 3 + 2 + 0 = 5, no carry
        a=4'd3; b=4'd2; cin=1'b0; #10;
        if ({cout,sum} !== 5'd5) begin
            $display("FAIL T1: 3+2+0 got=%b%b exp=00101", cout, sum);
            fail_count = fail_count + 1;
```

```
        end else $display("PASS T1: 3+2+0=%0d", {cout,sum});

        // Test 2: 7 + 8 + 0 = 15, no carry
        a=4'd7; b=4'd8; cin=1'b0; #10;
        if ({cout,sum} !== 5'd15) begin
            $display("FAIL T2: 7+8+0 got=%0d exp=15", {cout,sum});
            fail_count = fail_count + 1;
        end else $display("PASS T2: 7+8+0=%0d", {cout,sum});

        // Test 3: Overflow --- 15 + 1 + 0 = 16 -> cout=1, sum=0
        a=4'd15; b=4'd1; cin=1'b0; #10;
        if (cout!==1'b1 || sum!==4'd0) begin
            $display("FAIL T3: overflow cout=%b sum=%b", cout, sum);
            fail_count = fail_count + 1;
        end else $display("PASS T3: overflow cout=%b sum=%b", cout,
           sum);

        if (fail_count == 0)
            $display("\n=== ALL TESTS PASSED ===");
        else
            $display("\n=== %0d TEST(S) FAILED ===", fail_count);

        $finish;
    end

endmodule
```

## 6.3 Compilation and Execution

```
mkdir -p waves
iverilog -Wall -o adder4_sim adder4.v adder4_tb.v
./adder4_sim
gtkwave adder4.vcd &
```

Expected terminal output:

```
--- adder4 testbench ---
PASS T1:  3+2+0=5
PASS T2:  7+8+0=15
PASS T3:  overflow cout=1 sum=0000

=== ALL TESTS PASSED ===
```

# 7 Common Errors and How to Debug Them

## 7.1 Compilation Errors

| Error Message | Likely Cause and Fix |
|---|---|
| Unknown module type: ALU | The ALU.v file was not included in the iverilog command. Add it: `iverilog ... src/ALU.v ...` |
| variable X is declared twice | Duplicate wire or reg declaration in the same module. Remove one. |
| net X has conflicting declaration | X declared as both wire and reg. Pick one: `reg` if assigned in `always`, `wire` if driven by `assign`. |
| Port width mismatch | A wire of the wrong width was connected to a port. Check all port connections. |
| syntax error, expecting 'endmodule' | Missing `endmodule` at the end of file, or missing `begin/end` pair. |
| `include`: can not open file | The included file path is wrong. Use `-I src/` on the iverilog command. |

## 7.2 Simulation Errors and Unexpected Outputs

| Symptom | Likely Cause and Fix |
|---|---|
| Output shows X (unknown) | A wire is undriven. Check that all inputs to the DUT are driven before the first test. |
| Simulation hangs | No `$finish`, or `SIM_TIME` is very large. Add `$finish;` in testbench. |
| All outputs stuck at 0 | `always @(*)` has missing case branch. Add `$display` inside the block to trace. |
| PC does not advance | The `always @(posedge clk)` PC register is missing or `pc_next` is never assigned. |
| Branch not taken | Check `beq_taken` / `bne_taken` logic. |
| `$readmemb`: cannot open file | Run simulation from `test/` directory: `cd test && ../build/sim` |

## 7.3 Debugging Strategies

**Use $monitor for continuous signal tracing**

Add this inside your integration testbench's initial block:

```
$monitor("%0t | PC=%0h | instr=%b | R0=%0d R1=%0d R2=%0d",
    $time, uut.DU.pc_current, uut.DU.instr,
    uut.DU.reg_file.reg_array[0],
    uut.DU.reg_file.reg_array[1],
    uut.DU.reg_file.reg_array[2]);
```

**Use hierarchical signal references**

From the testbench, read any internal signal using dot notation:

```
uut.DU.pc_current                // PC inside Datapath inside StarCore1
uut.DU.reg_file.reg_array[3]     // R3 inside GPR inside Datapath
uut.DU.alu_result                // ALU output wire inside Datapath
uut.CU.reg_write                 // reg_write signal inside ControlUnit
```

**Isolate the fault to a single module**

If the integration test fails, run each component testbench individually to confirm it passes before re-testing integration.

# 8 Quick Reference Card

## 8.1 Key Commands

| Task | Command |
|---|---|
| Compile single module | `iverilog -Wall -o sim_out src/ALU.v tb/ALU_tb.v` |
| Compile with include path | `iverilog -Wall -I src/ -o sim src/ALU.v tb/ALU_tb.v` |
| Compile full integration | `iverilog -Wall -I src/ -o sim_star src/*.v tb/StarCore1_tb.v` |
| Run simulation | `./sim_out` |
| View waveforms | `gtkwave waves/out.vcd &` |
| Save GTKWave session | File > Write Save File in GTKWave |
| Reload session | `gtkwave waves/out.vcd signals.gtkw` |
| Kill hung simulation | Ctrl+C |
| Check iverilog version | `iverilog -v` |

## 8.2 Verilog Syntax Cheat-Sheet

| Construct | Syntax |
|---|---|
| Module declaration | `module Name (input a, output b); ... endmodule` |
| Wire declaration | `wire [15:0] mybus;` |
| Reg declaration | `reg [15:0] myvar;` |
| Memory array | `reg [15:0] mem [7:0];` |
| Continuous assign | `assign out = a & b;` |
| Combinational always | `always @(*) begin = end` |
| Clocked always | `always @(posedge clk) begin <= end` |
| Case statement | `case(sel) 2'b00: x=a; default: x=0; endcase` |
| If-else | `if (cond) ... else ...` |
| Sign extend 6→16 bit | `{{10{instr[5]}}, instr[5:0]}` |
| Concatenate | `assign y = {a[3:0], b[3:0]};` |
| Replicate | `assign y = {4{1'b0}};` |
| Instantiate sub-module | `Mod name (.p(w), .q(v));` |
| Include file | `include "../src/Parameter.v"` |
| Define macro | `define ROW_I 16` |

## 8.3 Assignment Rule Summary

| Situation | Assignment to use |
|---|---|
| Inside `always @(*)` — combinational logic | = (blocking) |
| Inside `always @(posedge clk)` — flip-flops | <= (non-blocking) |
| Outside `always` blocks — wires | `assign` (continuous) |
| Testbench stimulus | = (blocking, sequential) |